

Lab 04 — Questions

Question 1 [Diagnosis]

A developer runs `podman build -t api:1.0 .` from `~/projects/api/`, and their Dockerfile contains `COPY ../common/config.yml /app/`. The build fails with `possible escaping context directory error`. Explain why, and say why neither `-f`, nor an absolute path, nor `sudo` will change anything. What is the correct solution?

Question 2 [Analysis]

Under Docker, the build of an Angular project takes 4 minutes, of which 3 min 20 s are shown as `transferring context`. The folder contains `node_modules/` (900 MB) and `.git/` (200 MB). A colleague switches to Podman: the build now takes only 40 seconds, and they conclude that `.dockerignore` has become useless. Explain what was happening under Docker, what changed under Podman, and why they are wrong — naming the **second** risk, independent of slowness.

Question 3 [Understanding]

A Dockerfile contains `EXPOSE 8080`. The developer runs `podman run -d my-api:1.0` then notices that `curl http://localhost:8080` does not answer. They conclude the image is broken. Are they right? Explain the exact role of `EXPOSE`.

Question 4 [Analysis]

Compare these two Dockerfiles. What precisely does each do, and which one is correct for an API image?

```
# A
FROM docker.io/library/eclipse-temurin:21-jre
COPY api.jar /app/api.jar
RUN java -jar /app/api.jar
```

```
# B
FROM docker.io/library/eclipse-temurin:21-jre
COPY api.jar /app/api.jar
CMD ["java", "-jar", "/app/api.jar"]
```

Question 5 [Analysis]

Here are two images. For each, say what the commands `podman run img` and `podman run img --debug` produce.

```
# A
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
CMD ["--spring.profiles.active=prod"]
```

```
# B
CMD ["java", "-jar", "/app/api.jar", "--spring.profiles.active=prod"]
```

Then say which of the two still allows `podman run img sh` for debugging, and how to get out of it with the other.

Question 6 [Diagnosis]

A team complains that their redeployments always take ten seconds longer than expected, that Podman prints `resorting to SIGKILL` every time, and that Spring Boot never runs its *shutdown hooks*. The Dockerfile ends with:

```
CMD java -jar /app/api.jar
```

Diagnose, fix, and explain why that single line is enough to produce the symptom — and why the team does **not** observe it on their Alpine-based test image.

Question 7 [Analysis]

A Dockerfile needs the `JAVA_OPTS` variable at start-up:

```
ENV JAVA_OPTS="-Xmx512m"
ENTRYPOINT ["java", "$JAVA_OPTS", "-jar", "/app/api.jar"]
```

The container fails with `Unrecognized option: $JAVA_OPTS`. Explain, then give **two** possible fixes, stating what each costs.

Question 8 [Understanding]

A developer passes the database password at build time: `podman build --build-arg DB_PASSWORD=Secr3t! -t api:1.0 .`, explaining that an `ARG` does not persist in the image. Are they safe? Justify, and give the command that proves your answer.

Question 9 [Analysis]

A Maven Dockerfile is written like this:

```
COPY . /app
WORKDIR /app
RUN mvn -q package -DskipTests
```

Every build takes 6 minutes, even when a single `.java` file changed. Rewrite the instructions in the right order, explain the cache mechanism that makes your version faster, and say which build will remain slow anyway.

Question 10 [Diagnosis]

```
RUN apt-get update
RUN apt-get install -y curl vim
RUN rm -rf /var/lib/apt/lists/*
```

Name **three** distinct defects of these three lines, and give the correct version.

Question 11 [Analysis]

After modifying only the last `COPY` of their Dockerfile, a developer sees the build print `--> Using cache` on the first eight steps then rebuild the last two. The next day, they add an `ENV` variable in

third position and the whole build starts from scratch. Explain both behaviours with the same rule.

Question 12 [Understanding]

`COPY` and `ADD` look equivalent. Give two behaviours specific to `ADD`, explain why the official recommendation is to use `COPY`, and name the only case where `ADD` remains justified.

Question 13 [Analysis]

A Dockerfile contains `USER 1000:1000` right after the `FROM`, before the `COPY` and `RUN apt-get install` instructions. The build fails with `Permission denied`. Explain, and say where to place `USER` in a well-written Dockerfile. Then: `rootless`, `podman top` shows for that container `USER 1000` and `HUSER 100999`. Why that second number, and is the `USER` instruction still useful since "root" is already not root?

Question 14 [Analysis]

Your colleague claims: "You must reduce the number of layers as much as possible, so put everything in a single `RUN`." Discuss: in which cases are they right, in which cases does that rule hurt build time and transfer weight?